



CREACIÓN DE UNA IMAGEN DOCKER

CREACIÓN DE UNA IMAGEN DOCKER.....	1
1.  Características principales de Docker.....	2
1.1. Arquitectura Docker.....	2
1.1.1. Image (Docker Image).....	3
1.1.2. Container.....	3
2.  Instalación de Docker Engine en la máquina host.....	4
2.1.  Debian, Ubuntu, Linux Mint.....	4
2.2.  Fedora y distribuciones RHEL-family.....	6
3.  Creación de un contenedor.....	7
3.1. Bind mounts vs. volumes.....	7
3.2. Carpeta de trabajo (bind mount).....	8
3.2.1.  Linux nativo.....	8
3.2.2.  WSL2.....	8
3.2.3.  macOS (Colima).....	9
3.3. Crear el contenedor con el comando docker run.....	9
3.3.1. Revisión de configuraciones.....	11
4.  Comandos docker run, docker start, attach.....	11
5.  Configuraciones adicionales en la máquina host.....	12
5.1. Ejecutar comandos de Docker sin sudo.....	12
5.2. Alias para docker attach, docker start.....	12
6.  Instalación de paquetes básicos en el contenedor.....	13
6.1.  Contenedor con Ubuntu.....	13
6.2.  Contenedor con CentOS Stream, Oracle Linux.....	13
6.2.1. Instalación de rlwrap.....	13
7.  Configuraciones adicionales en el contenedor.....	14
7.1. Variables de entorno.....	14
7.2. Zona horaria e idioma.....	15
7.2.1.  Zona e idioma en el contenedor con Ubuntu.....	15
7.2.2.  Zona e idioma en el contenedor con CentOS Stream, Oracle Linux.....	15
8.  Creación de usuarios en el contenedor.....	15
9.  Instalación del software de la base de datos.....	16
10.  Creación de imágenes a partir de un contenedor.....	20

Este documento es común a las cuatro asignaturas y se referencia desde la práctica 02

1. 🐳 Características principales de Docker

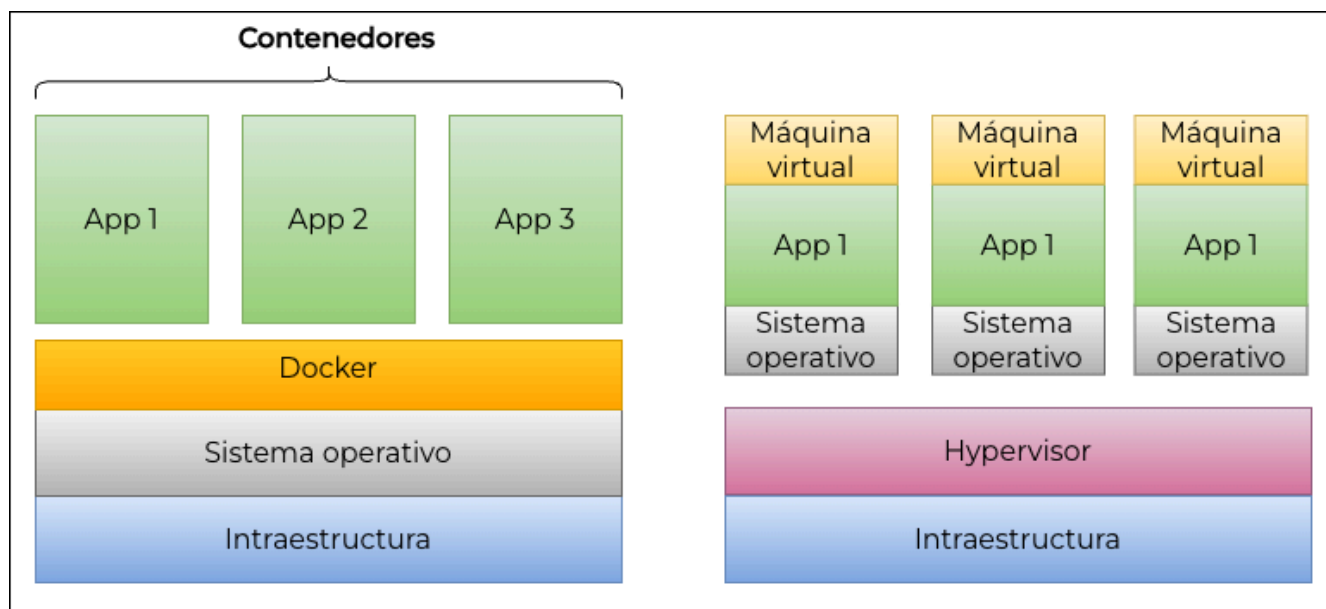
Docker es una plataforma que facilita el desarrollo, distribución, instalación y ejecución de aplicaciones haciendo uso del concepto de **contenedor de software**. Los contenedores permiten aislar, separar o desacoplar una aplicación de la infraestructura en la cual se ejecuta, creando un entorno independiente. Esto permite que múltiples contenedores se ejecuten simultáneamente en un mismo equipo de cómputo.

Un contenedor es un componente ligero que incluye lo necesario para que una aplicación funcione correctamente, es decir, es autosuficiente. Esta característica permite ejecutar aplicaciones sin depender del software o de las configuraciones existentes en la computadora, laptop o servidor.

Docker ofrece las herramientas y la infraestructura necesaria para administrar el ciclo de vida de los contenedores:

- Desarrollar una aplicación, sus componentes y dependencias.
- Representar una unidad de ejecución y distribución, útil para realizar pruebas de software.
- Instalar contenedores probados en entornos productivos.

A diferencia de las máquinas virtuales, los recursos de la máquina host no son reservados por cada contenedor — se comparten entre todos los contenedores. Esto permite que múltiples contenedores puedan compartir los recursos de una misma máquina.



1.1. Arquitectura Docker

Componente	Descripción
Arquitectura	Cliente-servidor.
Cliente Docker (docker)	Se comunica con un <i>Docker Daemon</i> , enviando comandos como docker build , docker pull , docker run .
<i>Docker Daemon</i> (dockerd)	Construye, ejecuta y distribuye los contenedores con base en los comandos del cliente.
Ubicación	Cliente y <i>Docker Daemon</i> pueden estar en la misma máquina o en máquinas diferentes.
<i>Docker Registries</i>	Almacenan <i>Docker Images</i> .
<i>Docker Images</i>	Plantilla que contiene todas las herramientas necesarias para ejecutar una aplicación en un contenedor.
<i>Docker Hub</i>	Sitio público para almacenar <i>Docker Images</i> ; es el registro por defecto.
Objetos Docker	<i>Images, containers, networks, volumes y plugins</i> .

1.1.1. Image (Docker Image)

- Representa una plantilla con instrucciones para crear un *Docker container*.
- Una imagen puede basarse en otra y agregarle características especiales. Por ejemplo, a partir de una imagen base de Ubuntu se puede crear una nueva que además instale un servidor web.
- Para crear una nueva imagen se usa un **Dockerfile**, que contiene las instrucciones para construirla y ejecutarla.

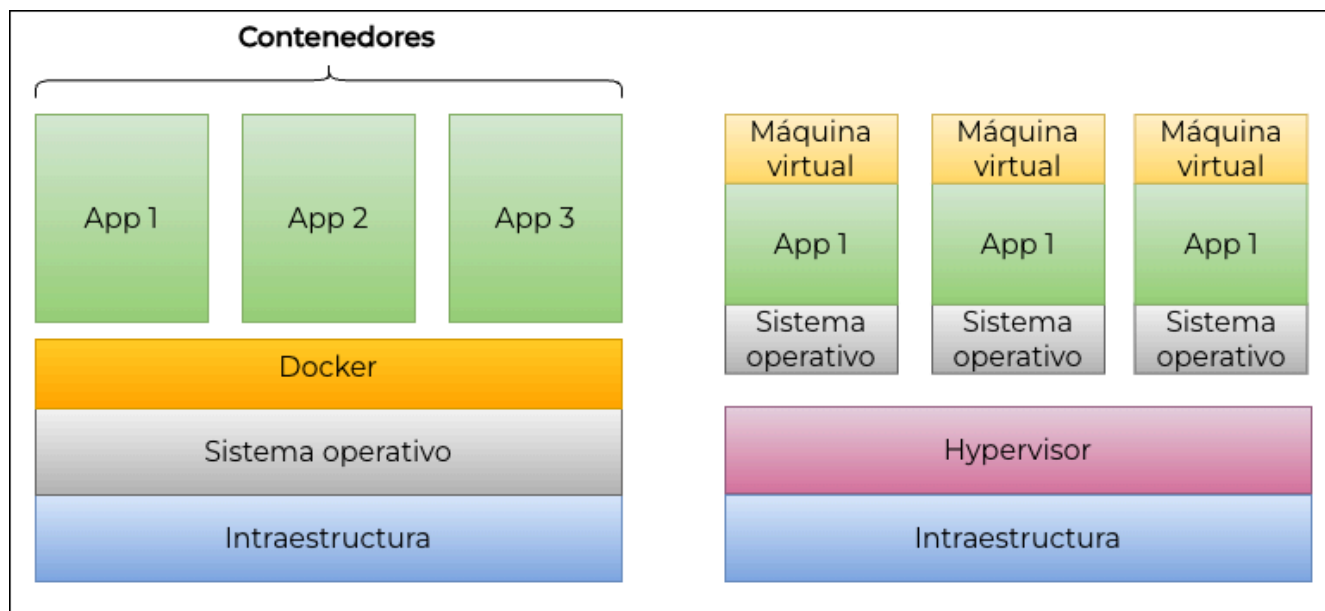
1.1.2. Container

- Representa una instancia ejecutable de una imagen.
- Se crea a partir de la definición de la imagen. Las operaciones básicas que se pueden aplicar son: **start**, **stop**, **move**, **delete**.

```
docker run -i -t ubuntu /bin/bash
```

Este comando realiza las siguientes acciones:

- En caso de no existir una imagen llamada **ubuntu**, Docker intentará obtenerla de Docker Hub (**docker pull ubuntu**).
- Se crea un contenedor a partir de la imagen (**docker container create**).
- Docker crea un sistema de archivos de lectura y escritura para el nuevo contenedor, así como una nueva interfaz de red.
- Las opciones **-i** (interactive) y **-t** (terminal) permiten interactuar con el contenedor mediante una terminal.
- Al ejecutar **exit** el contenedor se detiene, pero se conserva para iniciarse posteriormente.



De la imagen anterior, el cliente solicita la creación de un contenedor con `docker run`. Si la imagen existe de forma local, se genera el contenedor a partir de ella; de lo contrario, *Docker Daemon* la busca en un repositorio, típicamente **Docker Hub**.

2. Instalación de Docker Engine en la máquina host

Nota: esta sección aplica únicamente si la máquina host aún no cuenta con Docker instalado (Linux nativo o WSL2). En macOS, Docker ya quedó disponible mediante Colima desde la práctica 1 — no es necesario repetir ninguna instalación en el host. Para estos equipos, continuar con la sección "Creación de un contenedor".

Docker puede instalarse en prácticamente cualquier distribución Linux. Esta sección cubre las dos familias más comunes entre los estudiantes del curso: Debian/Ubuntu y Fedora. Para instalar Docker en cualquier otra distribución, usar este documento como guía y aplicar las instrucciones equivalentes de [Install Docker Engine](#).

Importante: no se hará uso de **Docker Desktop**; el objetivo del curso es practicar los comandos directos de [Docker Engine](#).

2.1. Debian, Ubuntu, Linux Mint

- A. Desinstalar versiones antiguas, en caso de existir.

```
sudo apt-get -y remove docker docker-engine docker.io containerd runc
```

- B. Instalar paquetes requeridos.

```
sudo apt-get -y install ca-certificates curl gnupg lsb-release
```

C. Crear el directorio de llaves e instalar la llave GPG del repositorio de Docker.

```
sudo mkdir -m 0755 -p /etc/apt/keyrings
sudo apt-get -y install curl

# El siguiente comando es multilínea; La barra invertida "\"" indica que la
#instrucción continúa en la siguiente línea
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

D. Agregar el repositorio de paquetes de Docker. Sustituir según la distribución.

i. Ubuntu:

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

ii. Linux Mint:

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(sed 's/UBUNTU_CODENAME=//;t;d' /etc/os-release) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Verificar que el archivo `docker.list` contenga una sola línea:

```
cat /etc/apt/sources.list.d/docker.list
```

E. Instalar Docker Engine.

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
sudo apt-get -y update
sudo apt-get -y install docker-ce docker-ce-cli containerd.io
sudo apt-get -y install docker-buildx-plugin docker-compose-plugin
```

F. Verificar la instalación.

```
sudo docker run hello-world
```

La salida debe incluir el mensaje `Hello from Docker!`.

2.2. 🎩 Fedora y distribuciones RHEL-family

- A. Desinstalar paquetes no oficiales previamente instalados.

```
sudo dnf -y remove docker \
docker-client docker-client-latest docker-common docker-latest \
docker-latest-logrotate docker-logrotate docker-selinux \
docker-engine-selinux docker-engine
```

- B. Configurar el repositorio de Docker.

```
sudo dnf -y install dnf-plugins-core
sudo dnf-3 config-manager --add-repo \
https://download.docker.com/linux/fedora/docker-ce.repo
```

- C. Instalar Docker Engine y Docker Compose.

```
sudo dnf -y install docker-ce docker-ce-cli containerd.io
sudo dnf -y install docker-buildx-plugin docker-compose-plugin
```

- D. Iniciar el servicio.

```
sudo systemctl enable --now docker
```

- E. Ajustar el parámetro `ulimit`. Por defecto, Fedora inicializa la cantidad de file descriptors en un valor muy elevado (1073741816), lo que puede provocar el error *library initialization failed - unable to allocate file descriptor table - out of memory*. Editar el archivo del servicio:

```
sudo nano /usr/lib/systemd/system/docker.service
```

Ubicar la línea `ExecStart` y agregar el límite de `ulimit`:

```
ExecStart=/usr/bin/dockerd --default-ulimit nofile=1024:65536 \
-H fd:// --containerd=/run/containerd/containerd.sock
```

Actualizar y reiniciar el servicio:

```
sudo systemctl daemon-reload
sudo systemctl restart docker
```

F. Verificar la instalación.

```
sudo docker run hello-world
```

3. Creación de un contenedor

Para crear un contenedor con Docker, se emplea el comando `docker run`. Para comprender los parámetros y valores que se emplean, revisar los siguientes [apartados](#).

3.1. Bind mounts vs. volumes

Docker ofrece dos mecanismos para que los datos de un contenedor persistan fuera de su capa de escritura: **bind mounts** y **volumes**. No son intercambiables — cada uno resuelve un problema distinto en el curso.

	Bind mount	Volume
Quién define la ruta	El usuario, con una ruta explícita del host	Docker, de forma interna
Sintaxis	<code>-v /ruta/host:/ruta/contenedor</code>	<code>-v nombre:/ruta/contenedor</code>
Portabilidad	Depende de que la ruta exista igual en cada máquina host	Igual en Linux nativo, WSL2 y Colima
Uso en el curso	Carpeta de trabajo compartida (\$UNAM_HOME)	Datos de la base de datos que se cree en la práctica 3

En esta práctica se usa un bind mount para compartir el material y los scripts entre la máquina host y el contenedor (\$UNAM_HOME). A partir de la práctica 3 se usará además un volumen para los datos de la base de datos que se cree — tanto para motores relacionales como NoSQL. Esto evita dos problemas: que los datos queden atrapados en la capa efímera del contenedor (se pierden si el contenedor se elimina), y que las imágenes de respaldo (`docker commit`) crezcan de forma innecesaria al incluir el peso completo de los datos.

Nota: un volumen persiste de forma independiente del contenedor. Eliminar y recrear el contenedor **no** reinicia el contenido del volumen — para partir de datos limpios es necesario eliminar y volver a crear el volumen explícitamente.

3.2. Carpeta de trabajo (bind mount)

Recordando de la práctica 1, se definió la variable de entorno `$UNAM_HOME` con una ruta dentro de la máquina host que contiene el material del curso. El valor recomendado es `/unam`. Dentro de los contenedores también se define la variable `$UNAM_HOME`. En este caso, su valor siempre será `/unam` de forma obligatoria.

De la práctica 1 también se explicó el mecanismo para poder compartir archivos entre la máquina anfitriona y la máquina host. Este procedimiento solo aplica a dicha práctica. A partir de esta práctica el mecanismo para compartir archivos con el contenedor cambia ligeramente y depende de la plataforma de la máquina host. Revisar la siguiente sección para identificar el mecanismo que aplica a cada plataforma.

3.2.1. Linux nativo

- `$UNAM_HOME` fue definida directamente en la máquina host, ya que ahí es donde corre Docker Engine. No existe separación entre "máquina host" y "máquina anfitriona": son el mismo entorno Linux.
- Es el caso más simple: cualquier archivo descargado o copiado dentro de `$UNAM_HOME` en la máquina host queda disponible de inmediato en el contenedor, sin pasos adicionales.
- Flujo recomendado: Descargar los archivos del curso directamente en `$UNAM_HOME/<asignatura>`, dentro de la práctica o tema correspondiente.

3.2.2. WSL2

- `$UNAM_HOME` fue definida directamente dentro de la distribución Linux de WSL2 (Ubuntu), ya que ahí es donde corre Docker Engine.
- El mecanismo para compartir archivos no es tan directo como en Linux nativo. Para ello, seguir las siguientes instrucciones:
 - Para compartir archivos con el contenedor, estos deben ubicarse dentro de `$UNAM_HOME` en la distribución Linux de WSL2. Sin embargo, al descargar archivos desde Windows, no es posible guardarlos en esta carpeta de forma directa.
 - Para descargar archivos desde Windows y que queden disponibles en el contenedor, se recomienda descargarlos primero en la carpeta `Downloads` del usuario de Windows y luego moverlos hacia `$UNAM_HOME` desde la terminal de WSL2:

```
mv /mnt/c/Users/<usuario>/Downloads/<archivo> $UNAM_HOME
```

- Otra opción es emplear la extensión [Remote - WSL](#) de Visual Studio Code, que permite abrir la carpeta de trabajo `$UNAM_HOME` directamente desde Windows y descargar los archivos ahí sin pasos adicionales.

3.2.3. 🍎 macOS (Colima)

- `$UNAM_HOME` fue definida en la máquina host, es decir, en la VM de Colima.
- A diferencia de WSL2, los comandos de Docker, por ejemplo `docker run`, se ejecutan desde la máquina anfitriona (macOS) y no desde la VM de Colima. Por ello, el uso directo de la máquina host en este entorno solo se utiliza en la práctica 1 y en algunas prácticas de BDA. Por esta razón `UNAM_HOME` fue definido en la VM (máquina host). Para cualquier otra práctica, no se hará uso de la VM de forma directa, y por lo tanto tampoco se hará uso de la variable `$UNAM_HOME` definida ahí.
- A diferencia de WSL2, una carpeta de la máquina anfitriona (macOS) se puede compartir directamente con el contenedor mediante un bind mount, similar a Linux nativo. Esto confirma aún más que el uso directo de la VM vía `colima ssh` no es necesario salvo los escenarios descritos anteriormente.
- Para compartir archivos con el contenedor, es necesario definir la variable `$UNAM_HOME` en la máquina anfitriona (macOS), ya que los comandos de Docker se ejecutan desde ahí.
- Para ello existen 2 detalles más:
 - El valor de `UNAM_HOME` no puede ser `/unam` ya que macOS no permite crear carpetas en la raíz del sistema de archivos. Se sugiere definirla como `$HOME/unam` (dentro del *home* del usuario). En esta carpeta es donde se pueden descargar los archivos del curso y compartirlos con el contenedor de forma directa. Crear esta carpeta en caso de no existir.
 - ¿Dónde se debe declarar `$UNAM_HOME`? En macOS no existe `/etc/profile.d/` como en Linux, por lo que se deberá definir en el archivo `$HOME/.zprofile`. ejecutar `source ~/.zprofile` para que la variable quede disponible en la sesión actual.
- Con estas configuraciones, cualquier archivo descargado o copiado dentro de `$UNAM_HOME` en la máquina anfitriona (macOS) queda disponible de inmediato en el contenedor, sin pasos adicionales.
- De aquí en adelante, cuando las instrucciones digan "en la máquina host" para el caso de macOS, se refieren a la máquina anfitriona — salvo que se indique explícitamente lo contrario (como en las prácticas de BDA que sí requieren entrar a la VM de Colima directamente).
- Para continuar con las siguientes secciones, crear la carpeta de la práctica dentro de `$UNAM_HOME` en la máquina anfitriona (macOS) y descargar ahí los archivos de la práctica:

```
mkdir -p $UNAM_HOME/<asignatura>/practicass/02
```

3.3. Crear el contenedor con el comando `docker run`

Este contenedor será creado a partir de una imagen base que contenga el sistema operativo requerido por la asignatura y compatible con las bases de datos empleadas en el

curso. Revisar la siguiente tabla para identificar la imagen base y el *tag* que corresponde a cada asignatura.

Asignatura	S.O. del contenedor	Imagen	Tag
BD, Lab BD	CentOS Stream	quay.io/centos/centos	stream9
BDA, BDD	Oracle Linux	container-registry.oracle.com/os/oraclelinux	9
BDN	Ubuntu	ubuntu	noble

Dentro de la carpeta de la práctica, crear un nuevo script `s-01-crea-contenedor.sh` con la instrucción para crear el contenedor. Ajustar los valores marcados en `<...>`.

```
#!/bin/bash
#@Autor(es):      Jorge Rodríguez
#@Fecha creación: dd/mm/yyyy
#@Descripción:    Crea un contenedor de Docker para la práctica 02

sudo docker run -i -t \
  -v $UNAM_HOME:/unam \
  --name c0-<so>-<iniciales> \
  --hostname h0-<so>-<iniciales>.fi.unam \
  --shm-size=2gb \
  <imagen>:<tag> bash
```

Descripción de los parámetros

Configuración	Descripción
<code>--name</code>	Nombre del nuevo contenedor.
<code><so></code>	Sistema operativo del contenedor (no confundir con el S.O. de la máquina host): <code>centos</code> para CentOS Stream, <code>ol</code> para Oracle Linux, <code>ubuntu</code> para Ubuntu.
<code><iniciales></code>	Primera letra del nombre y primera letra de cada apellido.
<code>--hostname</code>	Nombre de host del contenedor.
<code>-v</code>	Bind mount de la carpeta de trabajo. El destino dentro del contenedor es siempre <code>/unam</code> ; el origen es <code>\$UNAM_HOME</code> , según la plataforma (ver sección anterior).
<code><imagen>:<tag></code>	Nombre y la versión de la imagen base empleada para crear el contenedor. Como se mencionó anteriormente, sus valores dependen de la asignatura. Sustituir sus valores con base a la tabla anterior.
<code>bash</code>	Ejecuta el intérprete <code>bash</code> en modo interactivo dentro del contenedor.
<code>--shm-size</code>	Memoria compartida del contenedor (<code>/dev/shm</code>), usada para archivos temporales y comunicación entre procesos (IPC).

Al terminar de ajustar el script, ejecutarlo:

```
chmod 755 s-01-crea-contenedor.sh
./s-01-crea-contenedor.sh
```

3.3.1. Revisión de configuraciones

Después de ejecutar el comando con éxito, el usuario `root` queda en sesión dentro del contenedor. Verificar lo siguiente:

- A. Nombre del host (debe mostrar algo similar a `h0-01-jrc.fi.unam`):

```
cat /etc/hostname
```

- B. IP asignada, asociada al nombre de la máquina en `/etc/hosts`:

```
cat /etc/hosts
```

- C. Versión del sistema operativo:

```
cat /etc/os-release
```

- D. Salir del contenedor y regresar a la máquina host:

```
exit
```

4. Comandos `docker run`, `docker start`, `attach`

Nota: estos comandos se ejecutan en la máquina host (Linux nativo o WSL2). En el caso de macOS, se ejecutan desde la máquina anfitriona — ahí es donde vive el cliente `docker` instalado con Homebrew en la práctica 1; Colima solo aloja el *Docker Daemon*.

Cada `docker run` crea un nuevo contenedor. Para reutilizar un contenedor ya creado se emplean `docker start` (lo inicia en segundo plano) y `docker attach` (accede a él de forma interactiva). Ambos reciben el nombre o identificador del contenedor.

- A. Mostrar los contenedores existentes:

```
sudo docker ps -a
```

B. Iniciar un contenedor:

```
sudo docker start <nombre>
```

C. Acceder al contenedor:

```
sudo docker attach <nombre>
```

D. Iniciar y acceder en una sola instrucción:

```
sudo docker start <nombre> && sudo docker attach <nombre>
```

E. Abrir una terminal adicional hacia un contenedor ya iniciado:

```
sudo docker exec -it <nombre> bash
```

5. Configuraciones adicionales en la máquina host

5.1. Ejecutar comandos de Docker sin `sudo`

Nota: esta configuración no aplica en macOS — Colima expone el socket de Docker sin requerir `sudo` para el usuario que lo instaló.

A. Agregar al usuario administrador al grupo `docker` (creado durante la instalación de Docker), en Linux nativo o WSL2:

```
sudo usermod -aG docker <usuario_administrador>
```

B. Reiniciar la máquina host si la distribución lo requiere.

5.2. Alias para `docker attach`, `docker start`

Para evitar escribir los comandos completos cada vez, se recomienda crear los siguientes alias, sustituyendo `<so>` y `<contenedor>`:

```
alias docker<so>Base='docker start <contenedor> && docker attach <contenedor>'
alias docker<so>BaseT='docker exec -it <contenedor> bash'
```

Por ejemplo, para un contenedor con CentOS Stream:

```
alias dockerCentosBase='docker start c0-centos-jrc && docker attach c0-centos-jrc'
alias dockerCentosBaseT='docker exec -it c0-centos-jrc bash'
```

Estos alias se pueden agregar al final del archivo de variables de entorno de la máquina host:

- Linux nativo, WSL2: agregar al final del script `/etc/profile.d/99-custom-env.sh`.
- macOS: agregar al final del archivo `~/zprofile`

6. 📦 Instalación de paquetes básicos en el contenedor

El contenedor incluye un conjunto reducido de paquetes. Acceder como usuario `root` y ejecutar las instrucciones correspondientes.

6.1. 🐧 Contenedor con Ubuntu

```
apt update -y && apt upgrade -y
apt-get -y install sudo
apt-get -y install nano
apt-get -y install unzip
apt-get -y install iputils-ping
apt-get -y install locales
apt-get -y install lshw
apt-get -y install passwd
```

6.2. 🎩 Contenedor con CentOS Stream, Oracle Linux

```
dnf update -y
dnf install -y bc binutils compat-openssl11 elfutils-libelf fontconfig
dnf install -y glibc glibc-devel glibc-headers ksh libaio libasan liblsan
dnf install -y libX11 libXau libXi libXrender libXtst libxcrypt-compat
dnf install -y libgcc libibverbs libnsd librdmacm libstdc++ libxcb
dnf install -y libvirt-libs make policycoreutils policycoreutils-python-utils
dnf install -y smartmontools sysstat wget nano ncurses sudo perl
dnf install -y readline-devel passwd lshw hostname glibc-langpack-en
dnf install -y xterm iproute autoconf automake
dnf install -y gcc-gfortran
```

6.2.1. Instalación de `rlwrap`

Esta herramienta activa el historial de comandos en la línea de comandos de la base de datos.

- A. Visitar el [repositorio de rlwrap](#) y copiar el número de la última versión (etiqueta **latest**).
- B. Descargar el código fuente, sustituyendo `<version>`:

```
cd /tmp
wget https://github.com/hanslub42/rlwrap/archive/refs/tags/v<version>.zip
```

- C. Descomprimir:

```
unzip v<version>.zip
cd rlwrap-<version>
```

- D. Generar el script `configure` si no existe:

```
autoreconf --install
```

- E. Configurar, compilar e instalar:

```
./configure --without-libptytty
make
make install
```

- F. Comprobar la instalación:

```
rlwrap --version
```

7. Configuraciones adicionales en el contenedor

7.1. Variables de entorno

- Para contenedores con Oracle Linux o CentOS Stream, emplear el script `/etc/profile.d/99-custom-env.sh`.
- Para contenedores con Ubuntu (BDN), emplear `/etc/bash.bashrc`.

Nota: esta distinción no es arbitraria. Al acceder al contenedor (`docker run`, `docker exec`, `docker attach`) se obtiene una shell interactiva **sin** login, que nunca lee `/etc/profile` ni, por lo tanto, nada dentro de `/etc/profile.d/`.

En CentOS Stream y Oracle Linux, `/etc/bashrc` vuelve a ejecutar los scripts de `/etc/profile.d/` incluso en shells interactivas sin login — comportamiento

propio de Red Hat —, por lo que `/etc/profile.d/99-custom-env.sh` funciona igual.

En Ubuntu, `/etc/bash.bashrc` no hace ese recorrido; de ahí que se use directamente, o de lo contrario la variable nunca se cargaría con el flujo de acceso que se usa en el curso.

Configurar la variable `UNAM_HOME` con el valor `/unam` — este es siempre su valor dentro del contenedor, sin importar la plataforma de la máquina host, ya que es la ruta de destino del bind mount configurado al crear el contenedor.

7.2. Zona horaria e idioma

El sistema operativo del contenedor debe quedar configurado en idioma inglés y en la zona horaria de la Ciudad de México.

7.2.1. 🐧 Zona e idioma en el contenedor con Ubuntu

```
ln -fs /usr/share/zoneinfo/America/Mexico_City /etc/localtime
apt-get install -y tzdata
dpkg-reconfigure --frontend noninteractive tzdata
locale-gen en_US.UTF-8
echo "#idioma" >> /etc/bash.bashrc
echo "export LANG=en_US.UTF-8" >> /etc/bash.bashrc
echo "export LC_ALL=en_US.UTF-8" >> /etc/bash.bashrc
echo "export LANGUAGE=en_US.UTF-8" >> /etc/bash.bashrc
```

7.2.2. 🐧 Zona e idioma en el contenedor con CentOS Stream, Oracle Linux

```
echo "LANG=en_US.UTF-8" > /etc/locale.conf
echo "LANGUAGE=en_US" >> /etc/locale.conf
sudo ln -sf /usr/share/zoneinfo/America/Mexico_City /etc/localtime
```

8. 👤 Creación de usuarios en el contenedor

- A. Únicamente para contenedores con Ubuntu: la imagen incluye un usuario `ubuntu` que no se utilizará en el curso — eliminarlo.

```
userdel -r ubuntu
```

- B. Obtener la información del usuario administrador que se replicará en el contenedor:

```
id <usuario>
```

El `uid` y el `gid` obtenidos en la máquina host (Linux nativo, WSL2 o Colima) determinan los valores a usar en el contenedor.

C. Crear el mismo usuario y grupo en el contenedor con esos valores:

```
groupadd -g <gid> <usuario>
useradd -u <uid> -g <usuario> -m -d /home/<usuario> -s /bin/bash <usuario>
```

- `-g` asigna el grupo al que pertenecerá el nuevo usuario.
- `-m` crea el directorio *home* en caso de no existir.
- `-d` asigna el directorio *home*.
- `-s` asigna el intérprete `bash`.

D. Dar permisos de administrador al nuevo usuario:

- CentOS Stream, Oracle Linux — agregar al grupo `wheel`:

```
usermod -aG wheel <usuario>
```

- Ubuntu — agregar al grupo `sudo`:

```
usermod -aG sudo <usuario>
```

E. Asignar contraseña al nuevo usuario:

```
passwd <usuario>
```

F. Actualizar la contraseña de `root` (no viene configurada por defecto y se requiere durante la instalación del software de la base de datos):

```
passwd
```

Nota: se configuran dos contraseñas en el contenedor — una para `root` y otra para el usuario administrador. Tomar nota de ambas; son válidas únicamente dentro del contenedor.

9. Instalación del software de la base de datos

Esta sección aplica solo para **BD, BDA y BDD**. Para **BDN** se omite, ya que no se trabaja con Oracle. De ser el caso, continuar con la siguiente sección para realizar la creación de la imagen.

Se instala el software de Oracle sin crear ninguna base de datos — eso se realiza en la práctica 3. Ejecutar los siguientes pasos dentro del contenedor, como usuario **root**.

A. Crear los grupos requeridos a nivel de sistema operativo:

```
groupadd -g 54321 oinstall
groupadd -g 54322 dba
groupadd -g 54323 oper
groupadd -g 54324 backupdba
groupadd -g 54325 dgdba
groupadd -g 54326 kmdba
```

B. Crear el usuario **oracle**, dueño del software y de las bases de datos del servidor:

```
useradd -u 54321 -g oinstall -G dba,backupdba,dgdba,kmdba oracle
```

C. Asignar contraseña al usuario **oracle** (se sugiere **oracle** para efectos del curso):

```
passwd oracle
```

D. Crear el directorio de instalación y ajustar dueño, grupo y permisos:

```
mkdir -p /opt/oracle
cd /opt
chown oracle:oinstall oracle
chmod 755 oracle
```

E. Descargar el software de la base de datos. El paquete y la versión dependen de la **arquitectura** del contenedor (**uname -m**). Ejecutar los siguientes comandos dentro del contenedor, como usuario **root**:

-  Para sistemas **x86_64** (Linux nativo, WSL2) — Oracle Database Free 23ai:

```
cd /tmp
wget https://download.oracle.com/otn-pub/otn_software/db-free/oracle-database-free-23ai-23.8-1.e19.x86_64.rpm
wget https://yum.oracle.com/repo/OracleLinux/OL9/appstream/x86_64/getPackage/oracle-database-preinstall-23ai-1.0-2.e19.x86_64.rpm
```

- o 🍏 **aarch64** (macOS/Colima) — Oracle AI Database Free 26ai , solo se descarga un archivo.

```
cd /tmp
wget https://download.oracle.com/otn-pub/otn_software/db-free/oracle-ai-database-free-26ai-23.26.2-1.e19.aarch64.rpm
```

Nota: correr el software de Oracle bajo emulación x86_64 en Apple Silicon no es viable de forma confiable — se detectaron dos bugs de emulación sin solución posible desde nuestro lado: [ORA-00442](#) bajo Rosetta (bug conocido de Rosetta con la syscall que usa Oracle para su chequeo de instancia única) y [ORA-00443](#) bajo QEMU (bug de QEMU en modo usuario con `/proc/self/exe`). Por eso, en macOS se usa el RPM nativo **aarch64**, que no tiene build para 23ai en [e19](#) — solo a partir de 26ai. 26ai es 23ai con el *Release Update* de octubre 2025 aplicado (mismo producto, misma arquitectura interna); las diferencias frente a 23ai están concentradas en funcionalidad de IA, no en los mecanismos que cubre el curso. Por lo tanto, los estudiantes con Apple Silicon pueden usar 26ai sin problemas, aunque la práctica indique 23ai.

F. Realizar la instalación. Seleccionar alguna de las 2 siguientes opciones con base a la arquitectura y a la asignatura correspondiente.

- o 🐧🏠🍏 Para BDA, BDD en cualquier ambiente, y para BD 🐧🏠 que no hacen uso de macOS

```
export ORACLE_DOCKER_INSTALL=true
dnf install -y oracle-*.rpm
```

- o 🍏 Para BD que hace uso de macOS. Debido a que solo se instala un archivo RPM, se requiere un par de pasos extras ya que el segundo archivo RPM no está disponible en los repositorios para CentOS con arquitectura aarch64. Notar que la instalación del primer RPM indica que omita la inexistencia del segundo archivo RPM.

```
export ORACLE_DOCKER_INSTALL=true

#instala el primer RPM aunque no exista el segundo (nodeps)
rpm -ivh --nodeps oracle-ai-database-free-26ai-*.rpm

#Ejecutar también los 2 siguientes comandos
# 1. Crear y abrir el siguiente archivo
nano /etc/sysctl.d/98-oracle.conf
```

```
# 2. Incluir las siguientes configuraciones para el Kernel
fs.file-max = 6815744
kernel.sem = 250 32000 100 128
kernel.shmmni = 4096
kernel.shmall = 1073741824
kernel.shmmax = 4398046511104
fs.aio-max-nr = 1048576
net.ipv4.ip_local_port_range = 9000 65500
# 3. Aplicar los cambios. Si se observan mensajes como los siguientes,
# se pueden omitir: sysctl: permission denied on key "kernel..."
sysctl --system
```

G. Eliminar los archivos descargados para ahorrar espacio:

```
rm oracle-*.rpm
```

H. Verificar el cambio de sesión al usuario `oracle`:

```
su -l oracle
#si se obtiene el siguiente error, realizar el siguiente paso
#su: cannot open session: Permission denied
```

I. En caso de error, comentar las siguientes líneas en el archivo indicado y repetir el paso anterior. El nombre del archivo también depende de la arquitectura:

i.   x86_64 (Linux nativo/WSL2):

```
nano /etc/security/limits.d/oracle-database-preinstall-23ai.conf
```

ii.  aarch64 (macOS/Colima):

```
nano /etc/security/limits.d/oracle-ai-database-preinstall-26ai.conf
```

En ambos casos, comentar:

```
#oracle hard memlock 134217728
#oracle soft memlock 134217728
```

10. Creación de imágenes a partir de un contenedor

La imagen generada representa un respaldo: si el contenedor deja de funcionar, se puede crear uno nuevo a partir de la imagen sin repetir las configuraciones. También permite aplicar cambios que solo pueden hacerse al crear un nuevo contenedor (por ejemplo, agregar volúmenes o abrir un puerto).

A. Salir del contenedor:

```
exit
```

B. Generar la imagen a partir del contenedor:

```
docker commit c0-<so>-<iniciales> <so>-<iniciales>:1.0
```

- o `c0-<so>-<iniciales>` es el nombre del contenedor existente.
- o `<so>-<iniciales>` es el nombre de la nueva imagen.
- o `1.0` es el tag (versión) de la imagen.

Nota: Sustituir `<so>` con el nombre del sistema operativo instalado en el contenedor: `ol` (oracle linux), `centos` o `ubuntu` según corresponda.

C. Verificar las imágenes creadas:

```
sudo docker images
```

D. Con la imagen generada se pueden crear nuevos contenedores, agregando o modificando opciones. No es necesario ejecutar el siguiente comando por ahora; solo revisar la última línea, que hace uso de la imagen creada:

```
docker run -i -t \  
-v /unam:/unam \  
--name c1-<so>-<iniciales> \  
--hostname h1-<so>-<iniciales>.fi.unam \  
--expose 1521 \  
<so>-<iniciales>:1.0 bash
```

Todos los volúmenes deberán configurarse nuevamente en el nuevo contenedor. Una vez validado, los contenedores anteriores pueden eliminarse:

```
sudo docker rm <nombre_contenedor>
```

E. Para comprobar los volúmenes configurados en un contenedor:

```
docker ps -a --no-trunc --format "{{.Names}}\t{{.Mounts}}" | grep <nombre>
```

Continuar con las actividades de la práctica correspondiente.